

DevOps Lab

CMD Commands — Exam Reference Sheet

Program	Topic
Pgm 1	Verify Maven & Gradle Installation
Pgm 2	Create Maven Project & Build (Hello World)
Pgm 3	Create & Run Gradle Project (Groovy)
Pgm 4	Build Maven App then Migrate to Gradle
Pgm 5	Jenkins — Check Service & Get Admin Password
Pgm 6	Jenkins CI Pipeline with Maven (Freestyle Job)
Pgm 7	Ansible on Ubuntu/WSL — Install & Test Ping
Pgm 9	Azure DevOps — Account & Project Setup (Browser)
Pgm 10	Azure Pipelines — Push Maven Project via GitHub
Pgm 11	Azure Release Pipelines (Classic)

All the best for your exam! ■

Program 1 — Verify Maven & Gradle Installation

Step 1 — Verify Java is installed

```
java -version
```

■ Expected output: Displays java version (e.g. java version 21.0.x)

Step 2 — Verify Maven is installed

```
mvn --version
```

■ Expected output: Apache Maven 3.9.x Java version 1.8 or above OS name...

Step 3 — Verify Gradle is installed

```
gradle -v
```

■ Expected output: Gradle 8.12.x Kotlin: 2.0.x Groovy: 3.0.x JVM: 21.x

■ **Note:** If any command gives 'not recognized as an internal command', the tool is not installed or its bin folder is not added to the PATH environment variable. Re-check System Environment Variables (JAVA_HOME, MAVEN_HOME / M2_HOME, GRADLE_HOME) and ensure the bin sub-folder of each is added to PATH.

Program 2 — Create Maven Project & Build (Hello World)

Step 1 — Create a project folder and enter it

```
mkdir program2
cd program2
```

Step 2 — Generate a Maven project using the quickstart archetype

```
mvn archetype:generate ^
-DgroupId=com.program2.maven ^
-DartifactId=program2-example-jar ^
-DarchetypeArtifactId=maven-archetype-quickstart ^
-DinteractiveMode=false
```

■ Expected output: BUILD SUCCESS — project folder program2-example-jar is created

Step 3 — Navigate into the project and build (package)

```
cd program2-example-jar
mvn package
```

■ Expected output: BUILD SUCCESS — JAR file created inside target\ folder

Step 4 — Run the compiled Java application

```
java -cp target/program2-example-jar-0.0.1-SNAPSHOT.jar
com.program2.maven.App
```

■ Expected output: Hello World!

■ **Note:** The pom.xml is auto-generated inside program2-example-jar folder. Maven downloads dependencies from the internet on first run — make sure you have internet access. The version in the JAR name (0.0.1-SNAPSHOT) comes from pom.xml.

Program 3 — Create & Run Gradle Project (Groovy DSL)

Step 1 — Create a new folder and enter it

```
mkdir pgm3  
cd pgm3
```

Step 2 — Initialize a new Gradle project

```
gradle init
```

■ **Note:** Answer the prompts as follows: Select type of build → 1 (Application) Select implementation language → 3 (Groovy) Enter target Java version → 21 Project name → gradletest (or press Enter) Select application structure → 1 (Single application) Select build script DSL → 1 (Kotlin) Generate build using new APIs? → yes

■ **Expected output: BUILD SUCCESSFUL — Gradle project structure is ready**

Step 3 — Build and run the Gradle project

```
gradlew run
```

■ **Expected output: Hello World! followed by BUILD SUCCESSFUL**

Step 4 (Optional) — View project folder structure

```
tree
```

■ **Expected output: Tree of folders: .gradle app src build etc.**

Program 4 — Build Maven App then Migrate to Gradle

Step 1 — Create Maven project and enter it

```
mkdir pgm4
cd pgm4
mvn archetype:generate ^
-DgroupId=com.pgm4.test ^
-DartifactId=pgm4 ^
-DarchetypeArtifactId=maven-archetype-quickstart ^
-DinteractiveMode=false
cd pgm4
```

Step 2 — Build and run the Maven application first

```
mvn package
java -cp target/pgm4-0.0.1-SNAPSHOT.jar com.pgm4.test.App
```

■ Expected output: Hello World! (Maven run successful)

Step 3 — Migrate the Maven project to Gradle

```
gradle init
```

■ **Note:** Gradle detects the pom.xml automatically and asks: Found a Maven build. Generate a Gradle build? → yes Select build script DSL → 2 (Groovy) Generate build using new APIs? → yes Gradle converts pom.xml and creates build.gradle

■ Expected output: BUILD SUCCESSFUL — build.gradle created from pom.xml

Step 4 — Build and run with Gradle

```
gradle clean build
gradle run
```

■ Expected output: Hello World! Welcome to pgm4 BUILD SUCCESSFUL

■ **Note:** If you get 'mainClass not set' error, open build.gradle and add: application {
mainClass.set('com.pgm4.test.App') } Then run gradle clean build followed by gradle run again.

Program 5 — Jenkins — Check Service & Get Admin Password

Step 1 — Check if the Jenkins Windows service is running

```
sc query Jenkins
```

■ Expected output: STATE : 4 RUNNING (if already started)

Step 2 — Start Jenkins if it is stopped

```
net start Jenkins
```

■ Expected output: The Jenkins service was started successfully.

Step 3 — Read the initial admin password

```
type C:\ProgramData\Jenkins\.jenkins\secrets\initialAdminPassword
```

■ Expected output: A 32-character alphanumeric string like: a1b2c3d4e5f6...

Step 4 — Open Jenkins in the browser

```
start http://localhost:8080
```

■ **Note:** In the browser: 1. Paste the password from Step 3 into the Administrator password box → Continue 2. Click Install Suggested Plugins and wait for installation 3. Click Skip and continue as admin 4. Click Save and Finish → Start using Jenkins Jenkins dashboard is now ready at <http://localhost:8080>

Program 6 — Jenkins CI Pipeline — Freestyle Job with Maven

Step 1 — Make sure Jenkins is running

```
net start Jenkins
```

Step 2 — Find the full path of your pom.xml (copy this path)

```
cd pgm4  
echo %cd%\pom.xml
```

■ Expected output: Example: C:\Users\YourName\Desktop\pgm4\pom.xml

Step 3 — Open Jenkins in the browser

```
start http://localhost:8080
```

■ **Note:** In Jenkins (browser) — complete these steps: A. Manage Jenkins → Plugins → Available Plugins Search: Maven Integration → Install → wait B. Manage Jenkins → Tools Maven installations → Add Maven Name: MAVEN_HOME → Install automatically → Save C. Dashboard → New Item Enter name: MavenTest → Freestyle project → OK D. Scroll to Build Steps → Add build step Select: Invoke top-level Maven targets Maven Version: MAVEN_HOME Goals: clean install Click Advanced → POM: paste the path from Step 2 above E. Click Save → Build Now Click the build number → Console Output to see the log Expected: BUILD SUCCESS in the console output

Program 7 — Ansible on Ubuntu/WSL — Install & Test Connection

■ **Note:** IMPORTANT: All commands below must be run in an Ubuntu / WSL terminal, NOT in Windows CMD. Open Ubuntu from the Start menu or type wsl in CMD.

Step 1 — Update package list and install Ansible

```
sudo apt update  
sudo apt install ansible -y
```

■ Expected output: Ansible installed successfully with no errors

Step 2 — Verify Ansible version

```
ansible --version
```

■ Expected output: ansible 2.x.x config file = ... python version = ...

Step 3 — Create the Ansible configuration directory

```
sudo mkdir -p /etc/ansible
```

Step 4 — Create the inventory (hosts) file

```
sudo nano /etc/ansible/hosts
```

■ **Note:** Inside the nano editor type exactly these two lines: [local] localhost ansible_connection=local
Then press: Ctrl + X → Y → Enter to save and exit.

Step 5 — Check the inventory list

```
ansible-inventory --list -y
```

■ Expected output: all: children: local: hosts: localhost: ansible_connection: local

Step 6 — Test connection using the ping module

```
sudo ansible all -m ping
```

■ Expected output: localhost | SUCCESS => { "ping": "pong" }

Program 9 — Azure DevOps — Account Setup & Create Project

■ **Note:** This program is entirely browser-based. No CMD commands are needed. Follow the numbered steps below in Google Chrome.

Step 1 — Open the Azure for Students page

`https://azure.microsoft.com/en-us/free/students`

Step 2 — Click Start Free → No account? Create one

`Fill email, password, first name, last name, country, date of birth`

Step 3 — Verify your personal email

`A code is sent to your email – enter it on the Verify Email screen
→ Next`

Step 4 — Solve the robot puzzle

`Click Solve puzzle and complete the drag captcha`

Step 5 — Fill Academic Verification

`Enter your college email (e.g. name@college.ac.in) → Submit →
check that email for a verification link and click it`

Step 6 — Sign in to Azure Portal

`Go to https://portal.azure.com → sign in with your Microsoft
account`

Step 7 — Find Azure DevOps

In the search bar at the top type Azure DevOps → select Azure DevOps Organizations

Step 8 — Open your DevOps organization

Click My Azure DevOps Organizations → Continue

Step 9 — Create a new organization

Click Create new organization → name it → choose region: India → solve captcha → Continue

Step 10 — Create a project

Project name: e.g. maventestaz → Visibility: Public → Click + Create project

Program 10 — Azure Pipelines — Build Maven Project from GitHub

Step 1 — Enable Public Projects (in Azure DevOps browser)

```
Azure DevOps → Organization Settings → Policies  
Allow public projects → turn ON
```

Step 2 — Create Maven project (run in Git Bash or CMD)

```
mkdir maventest1  
cd maventest1  
mvn archetype:generate ^  
-DgroupId=com.dineshonjava ^  
-DartifactId=JavaHelloWorld ^  
-DarchetypeArtifactId=maven-archetype-quickstart ^  
-DinteractiveMode=false
```

■ Expected output: BUILD SUCCESS — JavaHelloWorld folder is created

Step 3 — Push the project to GitHub

```
cd JavaHelloWorld  
git init  
git add .  
git commit -m "azure pipeline example"  
git branch -M main  
git remote add origin  
https://github.com/YOUR_USERNAME/YOUR_REPO.git  
git push -u origin main
```

■ **Note:** Replace YOUR_USERNAME and YOUR_REPO with your actual GitHub username and repository name.

Step 4 — Create the pipeline (in Azure DevOps browser)

```
Project → Pipelines → New Pipeline
Where is your code? → GitHub
Sign in to GitHub and authorize Azure Pipelines
Select repository: YOUR_REPO
Configure your pipeline → Maven
Review YAML (mavenPomFile: 'JavaHelloWorld/pom.xml')
Click Save and Run → Run
```

■ Expected output: Pipeline runs and shows BUILD SUCCESS in the job log

■ **Note:** If you get a 'No hosted parallelism purchased' error, fill the Azure free parallelism request form and wait 1-2 days. The pipeline YAML file (azure-pipelines.yml) is automatically committed to your GitHub repository.

Program 11 — Azure Release Pipelines — Continuous Deployment

■ **Note:** Release pipelines are browser-based. You must first enable Classic pipelines in Organization Settings because Microsoft hides them by default.

Step 1 — Enable Classic Release Pipelines

```
Azure DevOps → Organization Settings  
→ Pipelines → Settings  
→ 'Disable creation of classic release pipelines' → turn OFF
```

■ **Expected output:** Releases option now appears in the left sidebar of your project

Step 2 — Create a Release Pipeline

```
Project → Pipelines → Releases → New → New release pipeline  
→ Select template: Empty job → Apply  
Stage name: Stage 1 → close the panel
```

Step 3 — Add an Artifact (link your Build pipeline)

```
Click + Add in the Artifacts box  
Source type: Build  
Project: your project  
Source (build pipeline): select the Maven pipeline from Pgm 10  
Default version: Latest → Add
```

Step 4 — Add a task to the Stage

Click '1 job, 0 tasks' link under Stage 1
Click + next to Agent job
Search: Maven → Add
Maven POM file: JavaHelloWorld/pom.xml
Goals: package
Save → OK

Step 5 — Create and deploy a Release

Click + Create release (top right)
Stages: leave as Automated
Click Create
Click the release link → Deploy → Deploy
Click Stage 1 → Logs to see execution

■ Expected output: Stage 1: Succeeded — all tasks show green ticks

■ **Note:** The Logs page shows each task: Initialize job, Download Artifacts, Maven (runs mvn package), Finalize Job — all should be green (Succeeded). You can click Download all logs to save a copy.

All the best for your DevOps Practical Exam! ■